

SOFT2201 - Week3 - Report: Rationalization of Design Decisions with Emphasis on SOLID Principles

August 17, 2024

Design Description

This report belows will prove that the design of a point-of-sale (POS) system for ComputersDirect will follow the SOLID principle, as represented in the provided UML diagram. The property that I focus mostly in the UML is that Single Responsibility. I have tried to split the tasks for the Customers and each interface will have the responsible for each task.

Single Responsibility Principle (SRP)

The SRP states that a class should have only one reason to change, meaning it should only have one responsibility. In the design:

- **Product, Desktop, Laptop, Accessories, and Service:** These classes are responsible for representing the different items sold in the store. Each class is responsible for encapsulating the specific attributes and behaviors of the products or services it represents.
- **Order:** This class handles the order-related operations such as viewing an order, managing products within an order, and calculating the price.
- **Payment method and its subclass ((Card, BankTransfer, GiftCard, Cash)):** These classes handle payment-related functionalities. Each payment method class is responsible for its specific payment process, ensuring that changes in one payment method do not affect others.
- **The Customer class** is implemented from two interfaces, Paying and InteractOrder. The reason for this design choice is to separate the responsibilities of the customer. Instead of grouping all methods into a single interface, the functionalities are divided into two main tasks that a customer typically performs: making payments and interacting with their orders. By doing this, the design ensures that the customer can manage payments and view or modify their orders, such as reviewing selected products, changing the payment method, or even canceling the order, all while adhering to the Single Responsibility Principle.

By dividing responsibilities across multiple classes, each class adheres to the SRP, making the system easier to maintain and extend.

Open/Closed Principle (OCP)

The OCP states that software entities should be open for extension but closed for modification. In the design:

- **PaymentMethod Interface:** This interface allows the system to add new payment methods without modifying existing code. For example, if a new payment method like Cryptocurrency needs to be added, it can implement the PaymentMethod interface without altering existing classes.
- **Product Class Hierarchy:** The Product class can be extended to include new product types like Tablets or Printers without modifying the existing Desktop, Laptop, or Accessories classes.

This design ensures that the system can evolve with minimal changes to existing code, adhering to the OCP.

Liskov Substitution Principle (LSP)

The LSP requires that objects of a superclass should be replaceable with objects of a subclass without affecting the correctness of the program. In the design:

- **Product Subclasses:** Objects of the subclasses Desktop, Laptop, and Accessories can be used wherever a Product is expected, and they will behave as intended. The methods in these subclasses do not violate the behavior expected by the Product class, ensuring compliance with the LSP.
- **PaymentMethod implementation:** Any class that implements the PaymentMethod interface, such as Card or Cash, can be used interchangeably without altering the system's behavior. This is crucial for maintaining a flexible and reliable payment processing system.

Dependency Inversion Principle (DIP)

The DIP states that high-level modules should not depend on low-level modules but on abstractions. In the design:

- **Payment Method Abstraction:** The Order class depends on the PaymentMethod interface rather than concrete classes like Card or Cash. This ensures that the Order class does not rely on specific payment methods and can interact with any class that implements the PaymentMethod interface.
- **Order management interface:** The Order class interacts with the InteractOrder interface, which abstracts the order management functionality. This allows for different implementations of order management without altering the Order class.

This use of abstraction makes the system more flexible and easier to extend or modify, adhering to the DIP.

Interface Segregation Principle (ISP)

The ISP states that clients should not be forced to depend on interfaces they do not use. In the design:

- **Separate Interfaces for Payment and Order Management:** The interfaces InteractOrder and Paying are clearly separated. InteractOrder is responsible for order-related operations, while Paying deals with payment processes. This segregation ensures that classes implementing these interfaces are not burdened with unnecessary methods, adhering to the ISP.

- **Specific Payment Method Interface:** The Validate interface is used for specific payment methods that require validation, such as GiftCard or Card. This ensures that only relevant methods are implemented by the classes that need them.

Conclusion

The design of the point-of-sale system for ComputersDirect adheres to the SOLID principles, ensuring a robust, maintainable, and scalable architecture. By following these principles, the system is well-prepared to accommodate future changes and expansions, such as the addition of new product types, payment methods, or order management features, without significant modifications to existing code.

This approach provides a strong foundation for building a reliable and user-friendly point-of-sale system that meets the needs of both the customers and the business.